⭐ MANAGING LARGE PLAYBOOKS:

- When a playbook gets long or complex, you can divide it up into smaller files to make it easier to manage.
- You can combine multiple playbooks into a main playbook modularly, or insert lists of tasks from a file into a play. This can make it easier to reuse plays or sequences of tasks in different projects.
- You can use following directives to manage large playbooks and increase reusability.

Directive	Purpose	Static/Dynamic	Allowed Level	Hosts Defined Where
import_playbook	Bring in entire playbooks	Static	Top level only	Inside imported playbook
import_tasks	Bring in task lists	Static	Inside a play	In main play
include_tasks	Dynamically load task lists	Dynamic	Inside a play	In main play

---

⭐ INCLUDING OR IMPORTING FILES:

- There are two operations that Ansible can use to bring content into a playbook. You can include content, or you can import content.
- When you include content, it is a dynamic operation. Ansible processes included content during the run of the playbook, as content is reached.
- When you import content, it is a static operation. Ansible preprocesses imported content when the playbook is initially parsed, before the run starts.

⭐ **IMPORTING PLAYBOOKS:**

- The import_playbook directive allows you to import external files containing lists of plays into a playbook.
- In other words, you can have a master playbook that imports one or more additional playbooks.
- Because the content being imported is a complete playbook, the import_playbook feature can only be used at the top level of a playbook and cannot be used inside a play.
- If you import multiple playbooks, then they will be imported and run in order.

EXAMPLE

```yaml
- name: Prepare the web server
  import_playbook: web.yml
- name: Prepare the database server
  import_playbook: db.yml
```

- **web.yml**

```yaml
- name: Configure web server
  hosts: webservers
  become: yes
  tasks:
    - name: Install NGINX
      package:
        name: nginx
        state: present
    - name: Ensure NGINX is running
      service:
        name: nginx
        state: started
        enabled: yes
```

db.yml

```yaml
- name: Configure database server
  hosts: dbservers
  become: yes
  tasks:
    - name: Install MariaDB
      package:
        name: mariadb-server
        state: present
    - name: Ensure MariaDB is running
      service:
        name: mariadb
        state: started
        enabled: yes
```

`import_playbook` ❌ You cannot define `hosts:` in the main playbook for imported playbooks. Each imported playbook must define its own `hosts:` internally. The `import_playbook` directive only works at the **top level** and merges complete playbooks, not plays.

🧠 **Can you include playbooks (not tasks)?**

No — **Ansible does not support include_playbook**.
Only **import_playbook** can bring in entire playbooks.

If You import playbooks, that are using variables, can you define those vars in main playbook?

Yes, you *can* define variables in the main playbook and pass them to imported playbooks — but only in a very specific way

```yaml
- name: Prepare the web server
  import_playbook: web.yml
  vars:
    package: nginx
    service: nginx

- name: Prepare the database server
  import_playbook: db.yml
  vars:
    package: mariadb-server
    service: mariadb
```

→

```yaml
- name: Install package
  package:
    name: "{{ package }}"
    state: present
```

You cannot rely on inventory variables to define which playbook to import

This fails:

```yaml
- import_playbook: "{{ playbook_name }}"
```

⭐ **IMPORTING AND INCLUDING TASKS**

- You can import or include a list of tasks from a task file into a play.
- A task file is a file that contains a flat list of tasks.

Example: webserver_tasks.yml :

```yaml
- name: Installs the httpd package
  yum:
    name: httpd
    state: latest
- name: Starts the httpd service
  service:
    name: httpd
    state: started
```

- You can statically import a task file into a play inside a playbook by using the import_tasks feature.
- When you import a task file, the tasks in that file are directly inserted when the playbook is parsed.
- The location of import_tasks in the playbook controls where the tasks are inserted and the order in which multiple imports are run.

Example_import_task:

```yaml
- name: Install web server
  hosts: webservers
  tasks:
    - import_tasks: webserver_tasks.yml
```

- When you import a task file, the tasks in that file are directly inserted when the playbook is parsed.
- Because import_tasks statically imports the tasks when the playbook is parsed, there are some effects on how it works:

- When using the import_tasks feature, conditional statements set on the import, such as when, are applied to each of the tasks that are imported.

    ```yaml
    - import_tasks: webserver_tasks.yml
      when: ansible_os_family == "RedHat"
    ```

Ansible **parses** the imported file *before* execution (static import).

Because of this, the **`when:`** condition is applied to **every task inside the imported file**, as if you had written:

```yaml
- name: Install httpd
  yum:
    name: httpd
    state: latest
  when: ansible_os_family == "RedHat"
- name: Start httpd
  service:
    name: httpd
    state: started
  when: ansible_os_family == "RedHat"
```

- You cannot use loops with the import_tasks feature.
- if you use a variable to specify the name of the file to import, then you cannot use a host or group inventory variable.
- You can also dynamically include a task file into a play inside a playbook by using the include_tasks feature.

Example_include_task

```yaml
- name: Install web server
  hosts: webservers
  tasks:
    - include_tasks: webserver_tasks.yml
```

- The include_tasks feature does not process content in the playbook until the play is running and that part of the play is reached.
- The order in which playbook content is processed impacts how the include tasks feature works.
- When using the include_tasks feature, conditional statements such as when set on the include determine whether or not the tasks are included in the play at all.

  Meaning:

  - If the condition is true → the task file is loaded, and all tasks inside it run normally.
  - If the condition is false → the task file is not included at all, and none of its tasks exist in the play.
- If you run ansible-playbook --list-tasks to list the tasks in the playbook, then tasks in the included task files are not displayed. The tasks that include the task files are displayed.
- By comparison,the import_tasks feature would not list tasks that import task files, but instead would list the individual tasks from the imported task files.
- You cannot use ansible-playbook --start-at-task to start playbook execution from a task that is in an included task file.

- You cannot use a notify statement to trigger a handler name that is in an included task file.

  Include-task.yml

  ```yaml
  - name: Install httpd
    yum:
      name: httpd
      state: present
    notify: restart_httpd
  handlers:
    - name: restart_httpd
      service:
        name: httpd
        state: restarted
  ```

  Main.yml.

  ```
  tasks:
    - include_tasks: web.yml
  ```

  ❌ **This will NOT work. Because - include_tasks: is dynamic, Handlers must be known at parse time, but included task files are loaded at runtime.**

  - ✓ **Handlers inside an imported task file WORK**

- You can trigger a handler in the main playbook that includes an entire task file, in which case all tasks in the included file will run

  Main.yml

  ```
  tasks:
    - include_tasks: web.yml
      notify: restart_services
  handlers:
    - name: restart_services
      service:
        name: httpd
        state: restarted
  ```

- You can create a dedicated directory for task files, and save all task files in that directory. Then your playbook can simply include or import task files from that directory. This allows construction of a complex playbook while making it easy to manage its structure and components.

⭐ DEFINING VARIABLES FOR EXTERNAL PLAYS AND TASKS:

- The incorporation of plays or tasks from external files into playbooks using Ansible's import and include features greatly enhance the ability to reuse tasks and playbooks across an Ansible environment.
- To maximize the possibility of reuse, these task and play files should be as generic as possible.
- Variables can be used to parameterize play and task elements to expand the application of tasks and plays.

```yaml
- name: Install web package
  package:
    name: nginx
    state: latest
- name: Starts the nginx service
  service:
    name: nginx
    state: started
```

Can be written as:

```yaml
- name: Install web package
  package:
    name: "{{ package }}"
    state: latest
- name: Starts the nginx service
  service:
    name: "{{ service }}"
    state: started
```

- **and it can be used as:**

```yaml
- name: Install and configure packages
  hosts: rhelnode
  tasks:
    - name: Import task file and set variables
      import_tasks: webserver_tasks.yml
      vars:
        package: firewalld
```

service: firewalld

OR --> u can use it as --> As per your requirements.

```yaml
- name: Import play file and set the variable
  import_playbook: play.yml
  vars:
    package: mariadb
```

- Ansible makes the passed variables available to the tasks imported from the external file.
- You can use the same technique to make play files more reusable. When incorporating a play file into a playbook, pass the variables to use for the play execution.

📘 Summary Table — Managing Large Playbooks in Ansible

Concept	Description	Key Points / Examples
Managing Large Playbooks	Divide long playbooks into smaller files for easier maintenance and reuse.	Use modular structure; combine multiple playbooks or task files.
Include vs Import	Two ways to bring external content into playbooks.	**Include:** Dynamic (processed during runtime). **Import:** Static (processed before execution).
Importing Playbooks	Use `import_playbook` to bring entire playbooks into a master playbook.	Must be at top level; runs imported playbooks in order. Example: `- import_playbook: web.yml`
Importing Tasks	Use `import_tasks` to statically insert tasks from a file.	Tasks are parsed before execution. Conditional (`when`) applies to all imported tasks. No loops allowed.
Including Tasks	Use `include_tasks` to dynamically load tasks during runtime.	Conditional (`when`) decides whether tasks are included. Tasks not shown in `--list-tasks` output. Cannot start execution from included tasks.

Concept	Description	Key Points / Examples
Task File Example	Flat list of tasks reused across plays.	`webserver_tasks.yml` with install/start service tasks.
Variables for External Plays/Tasks	Parameterize imported or included files for reuse.	Pass variables using `vars:`. Example: `import_tasks: webserver_tasks.yml vars: { package: firewalld, service: firewalld }`
Best Practice	Keep external task/play files generic and reusable.	Store task files in a dedicated directory; use variables for flexibility.